МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Техническое задание

«Разработка сервиса …»

Разработка мобильных приложений

(наименование дисциплины (модуля) в соответствии с учебным планом)

Уровень: бакалавриат

(бакалавриат, магистратура, специалитет)

Форма обучения: очная

(очная, очно-заочная, заочная)

Направление(-я) подготовки: 10.0

(код(-ы) и наименование(-я))

Институт: комплексной безопасности и специального приборостроения ИКБСП

(полное и краткое наименование)

Кафедра: КБ-99 «»

(полное и краткое наименование кафедры, реализующей дисциплину (модуль))

Выполнил: студент группы БИСО-99-99
Иванов Иван Иванович

Москва 2025

Назначение приложения

SafeCopy – мобильное приложение для Android, которое готовит «безопасную копию» документа, удостоверяющего личность, перед её передачей другому человеку или организации.

Проблема. В быту копии паспорта, СНИЛС или водительского удостоверения постоянно просят прислать фотографией в мессенджер или по почте: арендодатели, работодатели, гостиницы, автосалоны, курьерские службы. После отправки владелец больше не контролирует копию. Она остаётся в переписке и облачных резервных копиях получателя, может быть переслана дальше или попасть к злоумышленникам при взломе его аккаунта. Полное изображение документа с адресом регистрации, серией и номером используется мошенниками для регистрации аккаунтов, попыток оформить займ или составления фиктивных договоров.

При этом Федеральный закон № 152-ФЗ «О персональных данных» требует, чтобы обрабатываемые данные соответствовали заявленной цели и не были избыточными (ч. 5 ст. 5). На практике гражданин сам отправляет больше данных, чем нужно получателю, и не может доказать, кому именно передал копию, если она всплывёт в другом месте.

Решение. Приложение полностью на устройстве, без передачи данных в интернет:

– распознаёт документ и закрашивает поля, которые не нужны получателю;
– накладывает на всю площадь документа водяной знак с получателем, целью передачи, датой и уникальным кодом копии, поэтому копию нельзя использовать для другой цели;
– удаляет из файла метаданные (геолокацию, модель телефона, время съёмки);
– ведёт журнал отправок, по которому код с водяного знака указывает, кому была передана конкретная копия.

Целевая аудитория: физические лица, которые регулярно передают копии документов (аренда жилья, трудоустройство, бронирование, покупка автомобиля), а также люди, которым важна защита от мошенничества с персональными данными.

Основные требования к приложению

Приложение должно включать в себя:

1) Launch Screen с логотипом МИРЭА

Во время загрузки приложения (инициализация зашифрованного хранилища и модели распознавания текста) отображаются логотип МИРЭА и название приложения.

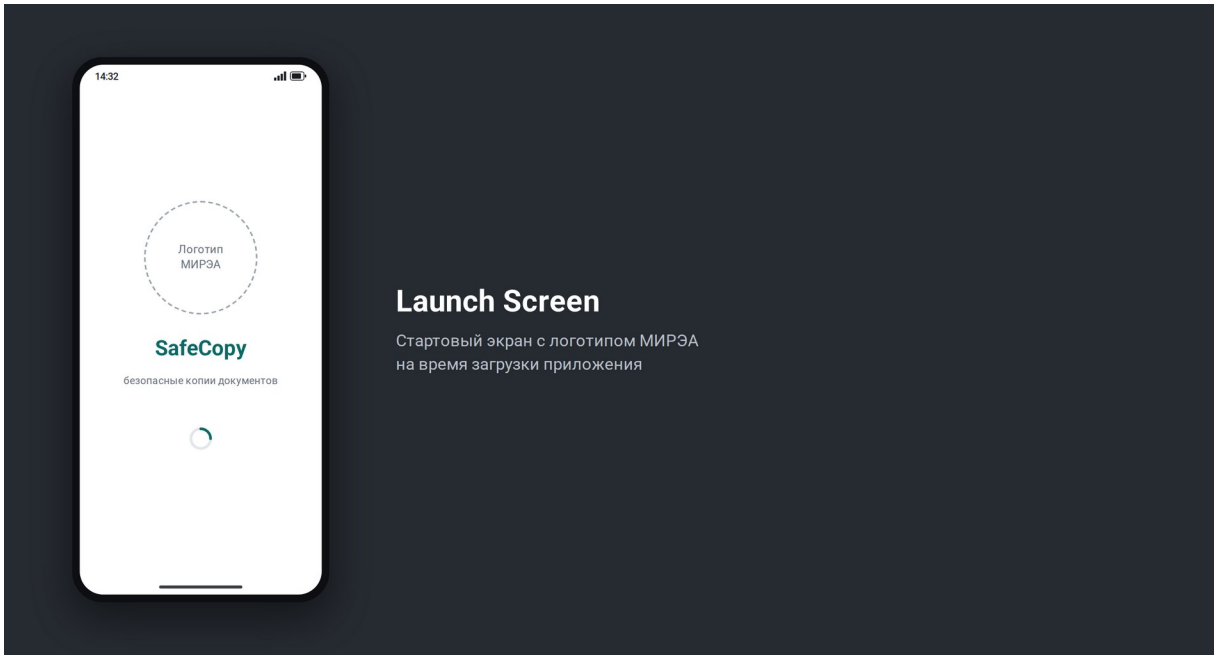


Рисунок 1 – Launch Screen

2) Вход в приложение по PIN-коду и биометрии

При первом запуске пользователь задаёт PIN-код из 4 цифр. При последующих запусках вход выполняется по отпечатку пальца или лицу (BiometricPrompt, класс Strong) либо по PIN-коду. Ключ шифрования хранилища генерируется в Android Keystore и становится доступен только после успешной аутентификации. После пяти неверных попыток ввод блокируется на 30 секунд.

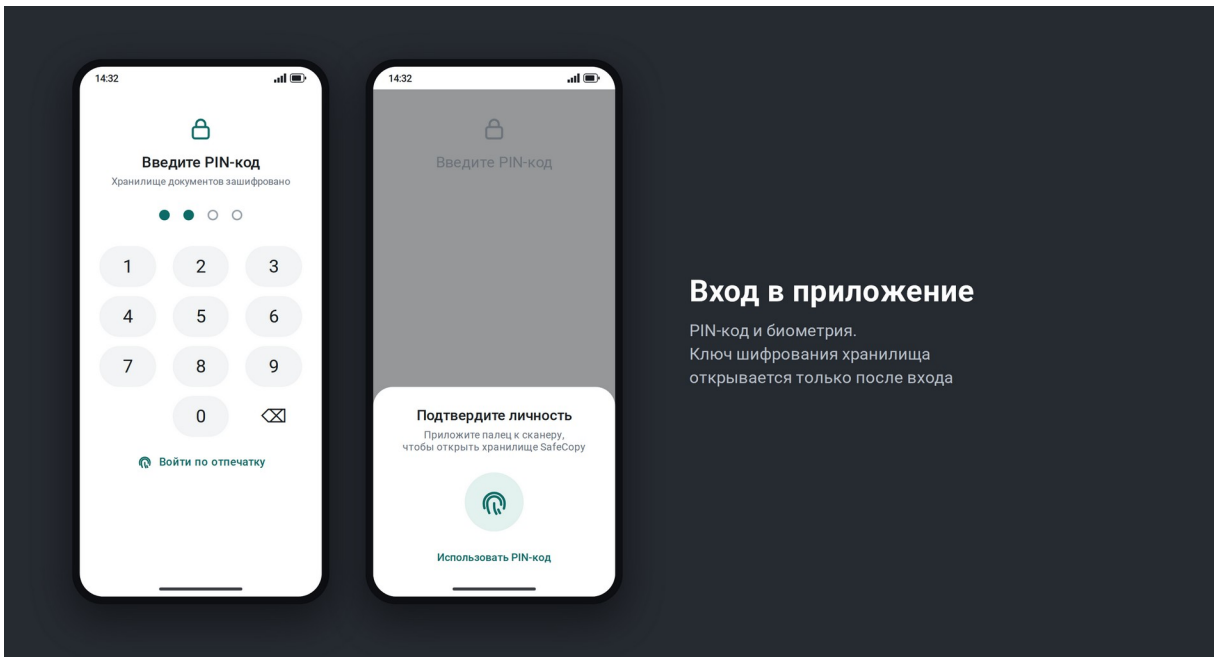


Рисунок 2 – Вход по PIN-коду и биометрии

3) Хранилище документов (главный экран)

Главный экран содержит список отсканированных документов: паспорт РФ, СНИЛС, водительское удостоверение, ИНН и другие. Для каждого документа отображаются количество страниц, дата обновления и число созданных копий. Документы хранятся только на устройстве в зашифрованном виде (AES-256-GCM). Внизу экрана расположена навигация между разделами «Документы», «Журнал» и «Настройки».

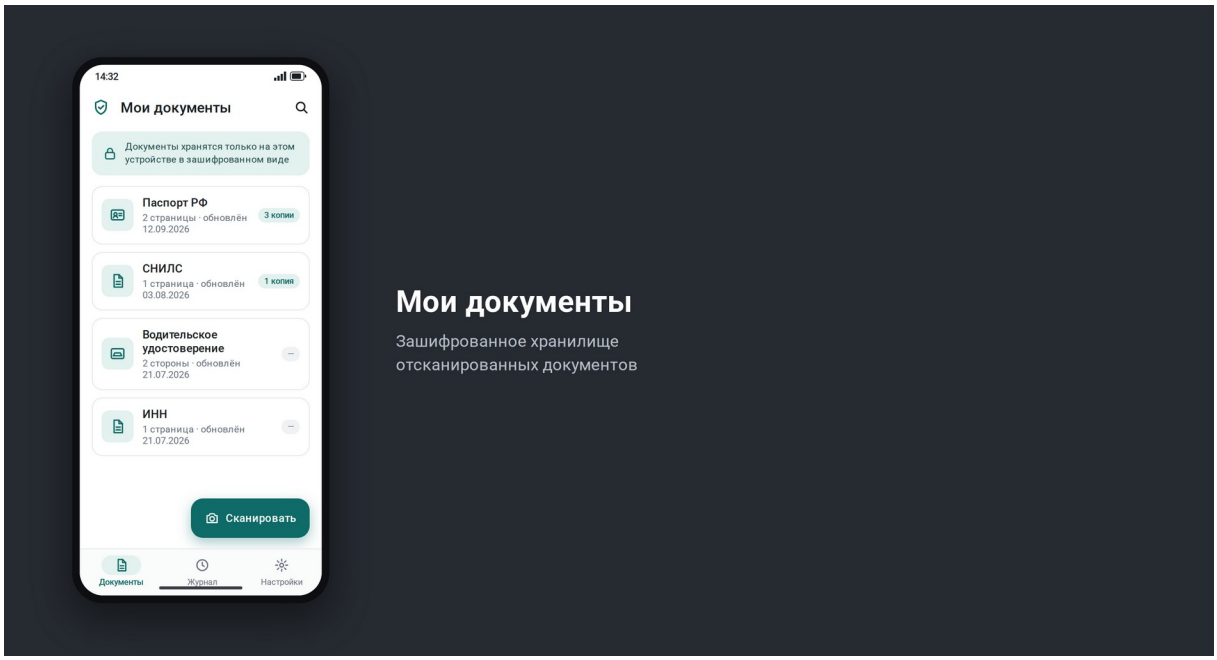


Рисунок 3 – Главный экран – хранилище документов

4) Съёмка документа камерой с индикатором наклона

Экран съёмки построен на CameraX. Приложение находит границы документа в кадре и выравнивает перспективу. Данные акселерометра (SensorManager, датчик TYPE_ACCELEROMETER) используются для индикатора наклона: снимок делается автоматически, когда границы найдены, а наклон телефона не превышает 3°. Это уменьшает искажения и повышает точность распознавания. Как альтернатива доступны импорт из галереи и системный сканер ML Kit Document Scanner.

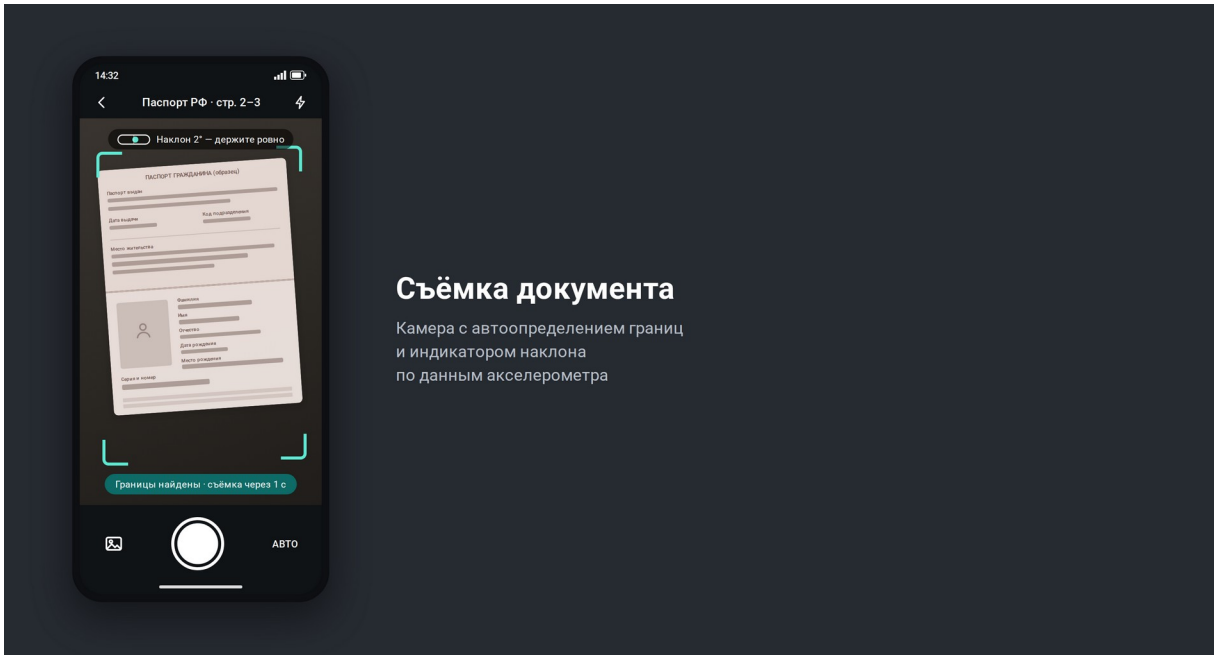


Рисунок 4 – Съёмка документа

5) Распознавание документа и маскирование полей

Приложение определяет тип документа и находит на изображении поля по шаблону разметки, уточняя их положение распознаванием текста (Tesseract, русский язык). Пользователь переключателями выбирает, какие поля скрыть: место жительства, серию и номер (полностью или с сохранением последних 4 цифр), код подразделения, место рождения. Произвольную область можно закрасить вручную. Маска выполняется сплошной заливкой, а не размытием, потому что размытие можно частично обратить.

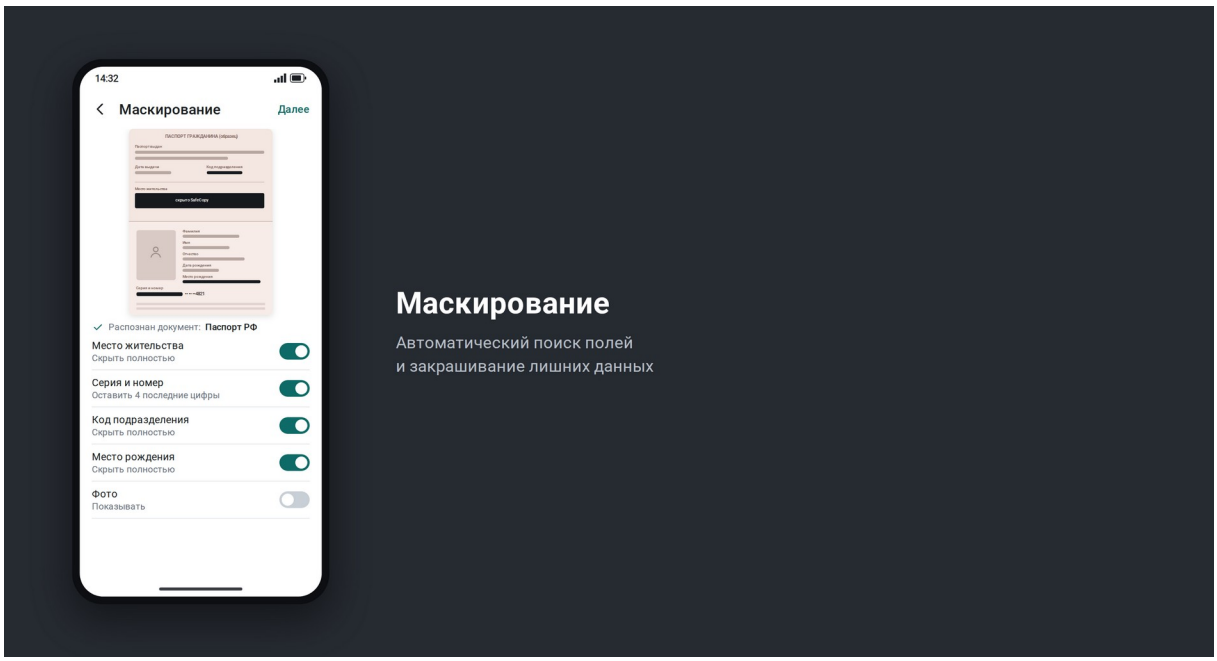


Рисунок 5 – Маскирование полей документа

6) Водяной знак с назначением копии

Пользователь указывает получателя (вручную или из контактов) и цель передачи. Дата подставляется автоматически, уникальный код копии (например, SC-7F3A) генерируется приложением. Текст водяного знака повторяется по диагонали на всей площади документа, прозрачность настраивается в диапазоне 20–60 %. Знак впечатывается в пиксели изображения, поэтому его нельзя удалить как отдельный слой.

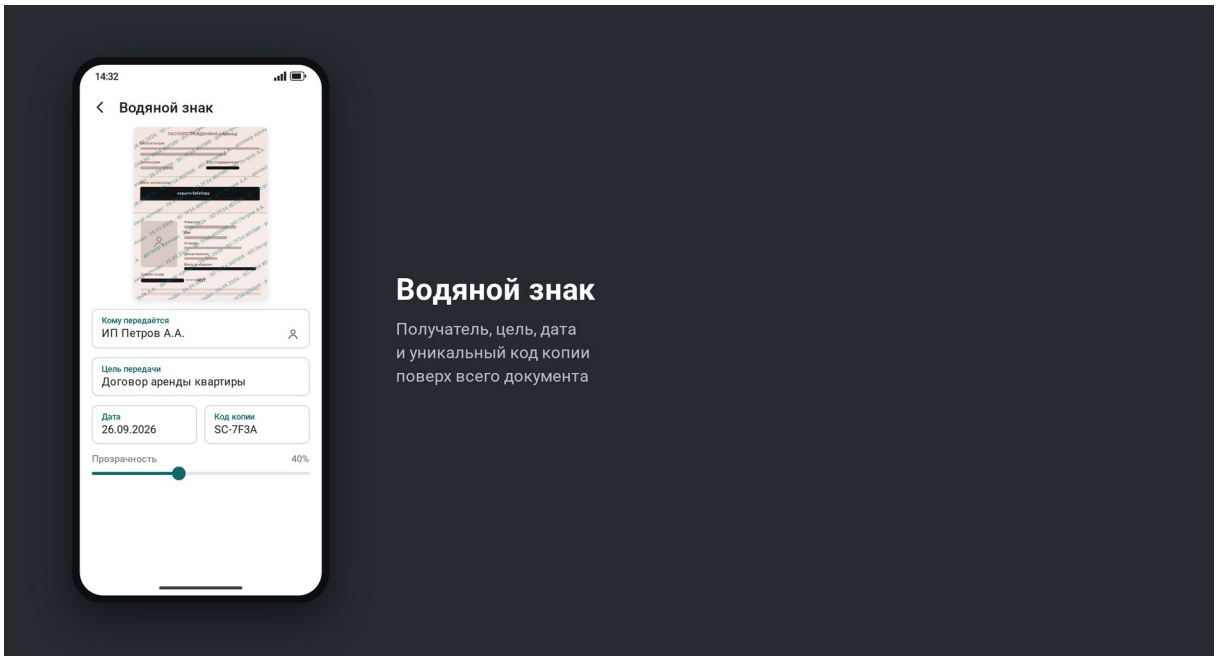


Рисунок 6 – Наложение водяного знака

7) Экспорт и отправка копии

Копия сохраняется в формате PDF или JPEG. Перед экспортом удаляются метаданные (EXIF: геолокация, модель телефона, время съёмки), по желанию понижается разрешение. Получатель выбирается через системный экран выбора контакта (ACTION_PICK), поэтому приложению не нужно разрешение на чтение всех контактов. Отправка выполняется через системное меню «Поделиться» (Android Sharesheet, ACTION_SEND и FileProvider). Временный файл удаляется после отправки.

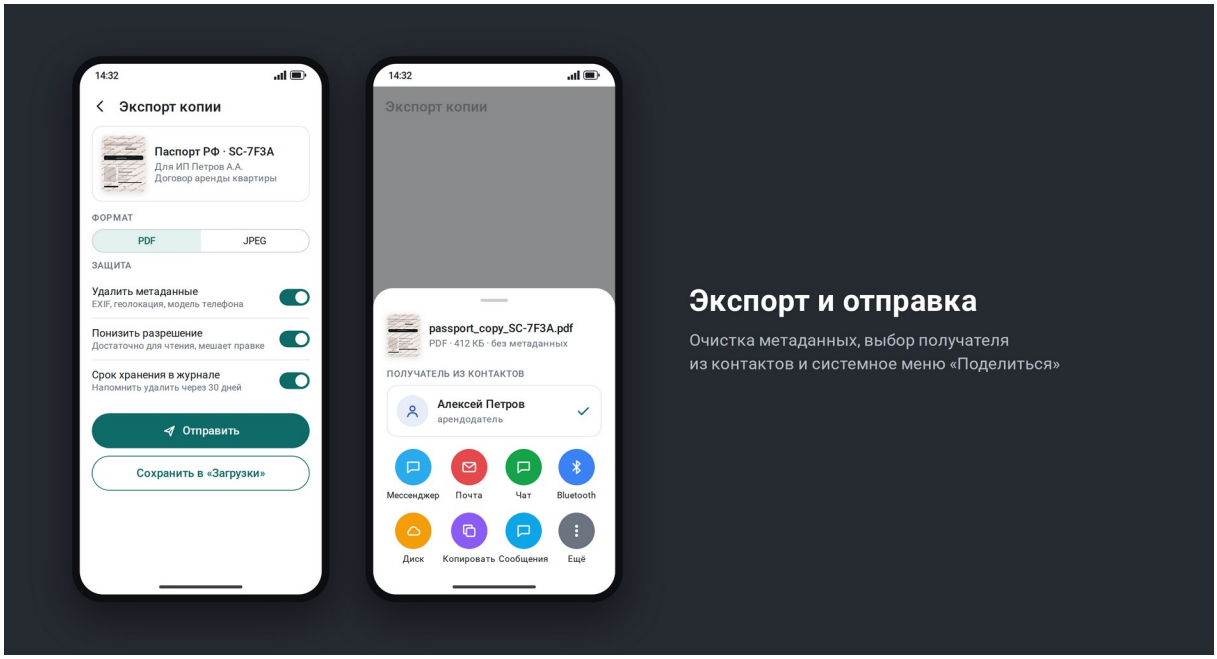


Рисунок 7 – Экспорт и отправка копии

8) Журнал отправок

Каждая отправка записывается в журнал: документ, получатель, цель, дата, канал передачи и код копии. Поиск по коду позволяет определить, кому была передана копия, если она обнаружится в другом месте. Через 30 дней после передачи (срок, установленный ч. 4 ст. 21 152-ФЗ для уничтожения данных после достижения цели обработки) приложение напоминает, что можно попросить получателя удалить копию.

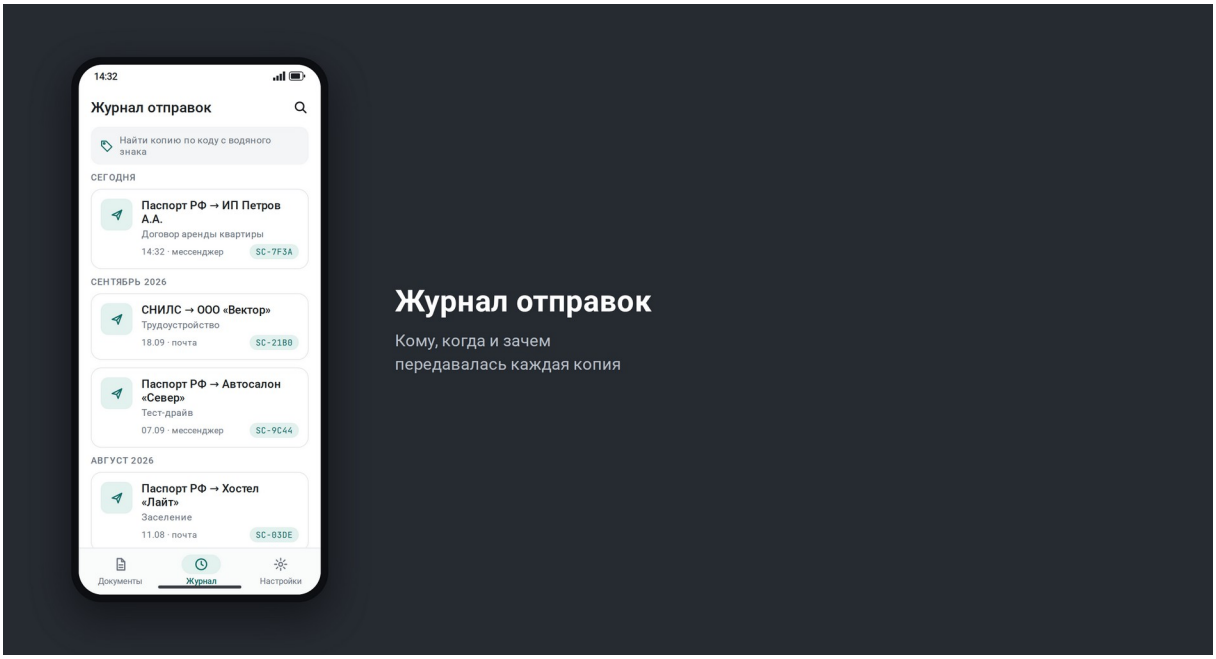


Рисунок 8 – Журнал отправок

9) Настройки безопасности

В настройках включаются вход по биометрии, время автоблокировки, запрет снимков экрана (FLAG_SECURE), скрытие содержимого окна в списке недавних приложений. Также можно изменить шаблон водяного знака, создать зашифрованную резервную копию хранилища и удалить все данные вместе с ключами шифрования. Приложение не запрашивает разрешение на доступ к интернету.

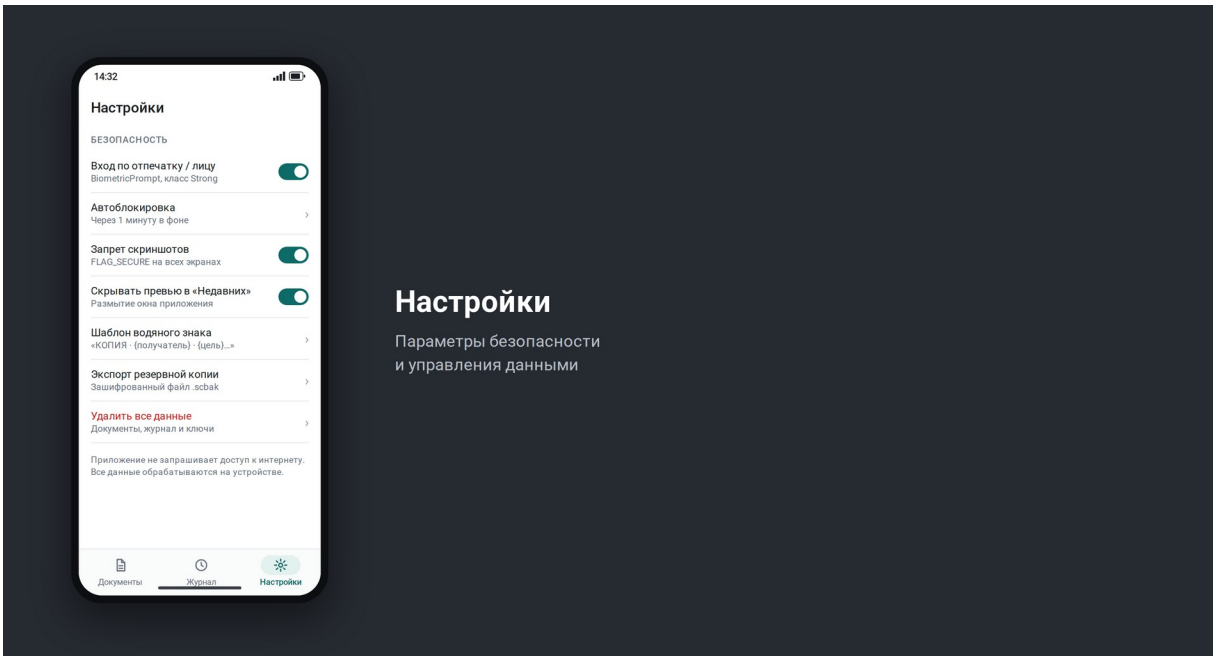


Рисунок 9 – Экран настроек

Описание приложения

Приложение разрабатывается на языке Kotlin для Android 8.0 (API 26) и выше, интерфейс строится на Jetpack Compose. При создании приложения используется шаблон проектирования MVVM.

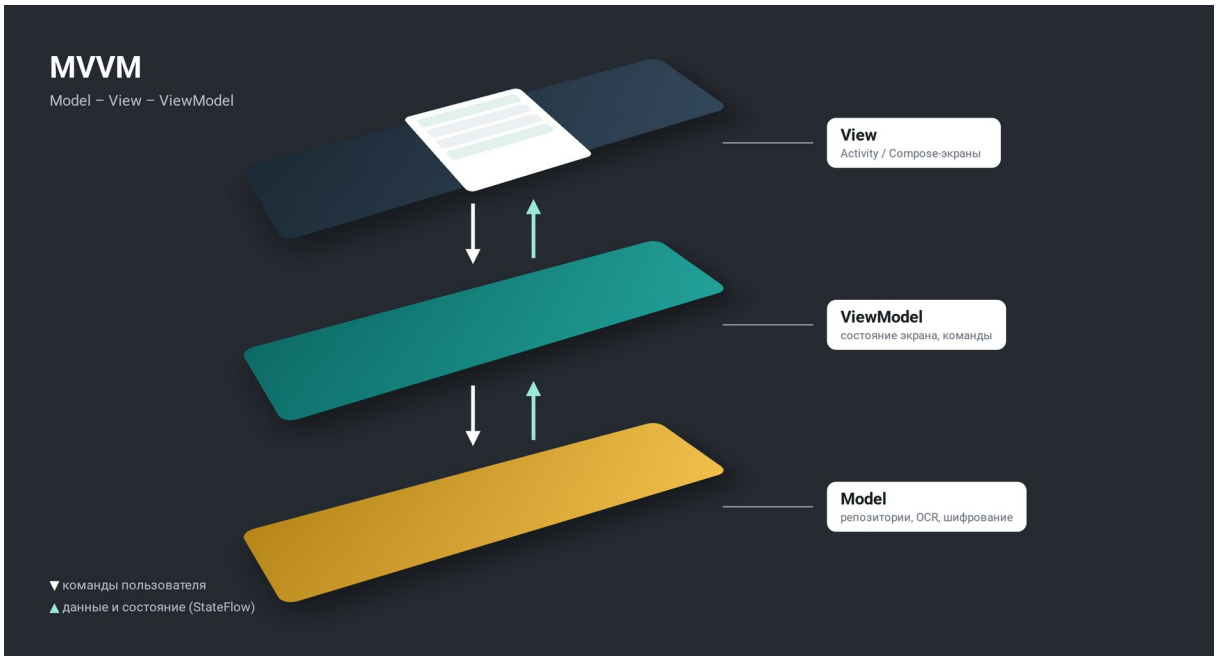


Рисунок 10 – Визуализация принципа работы MVVM

MVVM (Model – View – ViewModel) – шаблон проектирования пользовательского интерфейса, который отделяет бизнес-логику приложения (Model) от уровня отображения (View).

Model содержит бизнес-логику и работу с данными: репозитории документов и журнала отправок, модуль распознавания, обработку изображений и шифрование. View отображает состояние экрана и передаёт действия пользователя во ViewModel. ViewModel хранит состояние экрана в виде StateFlow, обращается к Model и переживает поворот экрана и смену конфигурации, поэтому процесс обработки документа не прерывается.

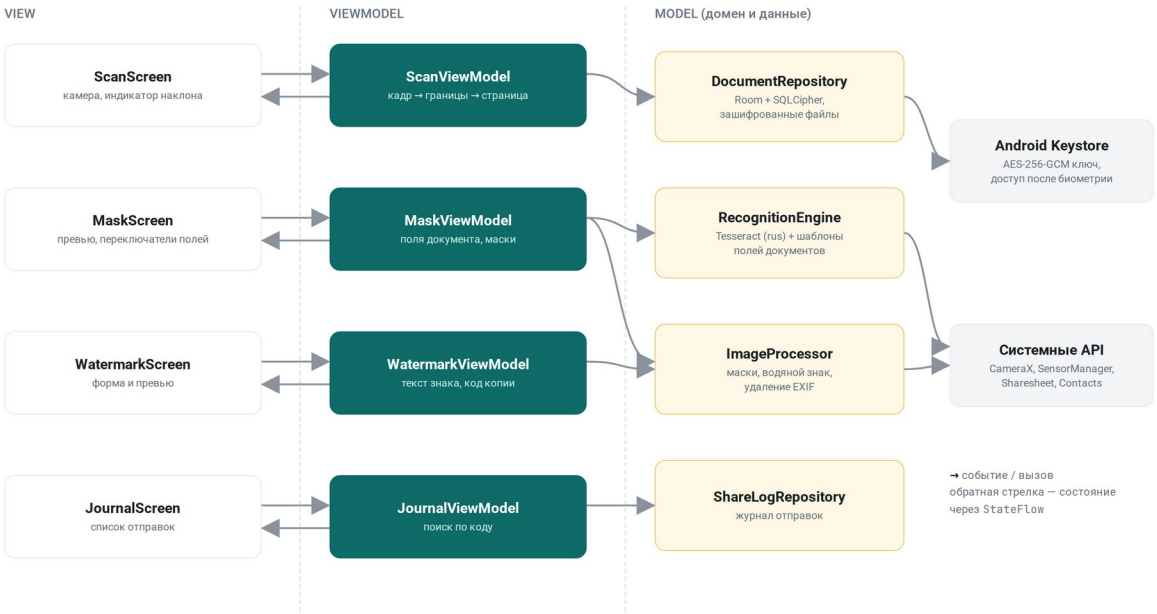


Рисунок 11 – Схема работы MVVM в приложении

Обработка документа выполняется последовательно и целиком на устройстве. Оригинал хранится в зашифрованном виде, а копия для отправки формируется во временном каталоге и удаляется после передачи.

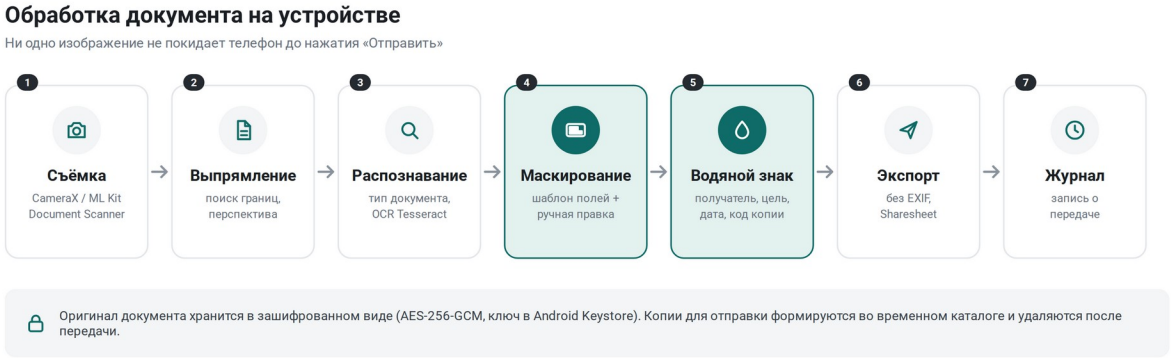


Рисунок 12 – Конвейер обработки документа

Использование API

При разработке приложения используются CameraX, ML Kit Document Scanner, Tesseract4Android, BiometricPrompt с Android Keystore, SensorManager, а также Room с SQLCipher для зашифрованной базы данных.

```
// build.gradle.kts (:app)
dependencies {
    // Камера и сканер документов
    implementation("androidx.camera:camera-camera2:1.4.2")
    implementation("androidx.camera:camera-lifecycle:1.4.2")
    implementation("androidx.camera:camera-view:1.4.2")
    implementation("com.google.android.gms:play-services-mlkit-document-scanner:16.0.0")
    // Распознавание текста (офлайн, русский язык)
    implementation("cz.adaptech.tesseract4android:tesseract4android:4.9.0")
    // Биометрия и зашифрованное хранилище
    implementation("androidx.biometric:biometric:1.1.0")
    implementation("androidx.room:room-ktx:2.6.1")
    implementation("net.zetetic:sqlcipher-android:4.5.6")
}
```

Рисунок 13 – Подключение библиотек

ML Kit Document Scanner

Используется как альтернативный режим съёмки: находит границы документа, выравнивает перспективу и возвращает готовые страницы. Обработка выполняется на устройстве через сервисы Google Play.

```
val options = GmsDocumentScannerOptions.Builder()
    .setGalleryImportAllowed(false)
    .setPageLimit(2)                        // разворот паспорта
    .setResultFormats(RESULT_FORMAT_JPEG)
    .setScannerMode(SCANNER_MODE_BASE)    // без облачных фильтров
    .build()

val scanner = GmsDocumentScanning.getClient(options)
scanner.getStartScanIntent(activity)
    .addOnSuccessListener { sender →
        scanLauncher.launch(IntentSenderRequest.Builder(sender).build())
    }
```

Рисунок 14 – Инициализация сканера документов

Tesseract4Android

Библиотека распознавания текста без подключения к интернету. Модель русского языка rus.traineddata поставляется вместе с приложением. Результат распознавания содержит текст и координаты блоков, по которым уточняется положение полей документа.

```
class RecognitionEngine(private val context: Context) {
    private val tess = TessBaseAPI()

    fun init(): Boolean {
        // rus.traineddata поставляется в assets и копируется в filesDir/tesseract/tessdata
        val dataPath = File(context.filesDir, "tesseract").absolutePath
        return tess.init(dataPath, "rus")
    }

    fun recognize(page: Bitmap): List<TextBlock> {
        tess.setImage(page)
        return tess.getResultIterator().toBlocks()   // текст + координаты
    }
}
```

Рисунок 15 – Инициализация модуля распознавания текста

BiometricPrompt и Android Keystore

Биометрическая аутентификация связана с криптографическим ключом: расшифровка хранилища возможна только с объектом Cipher, который система разблокирует после успешной проверки отпечатка или лица.

```
val promptInfo = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Подтвердите личность")
    .setSubtitle("Чтобы открыть хранилище SafeCopy")
    .setAllowedAuthenticators(BIOMETRIC_STRONG)
    .setNegativeButtonText("Использовать PIN-код")
    .build()

// ключ AES-256 в Android Keystore доступен только после аутентификации
val cipher = keyManager.getDecryptCipher()
biometricPrompt.authenticate(promptInfo, BiometricPrompt.CryptoObject(cipher))
```

Рисунок 16 – Вход по биометрии с разблокировкой ключа

Android Canvas

Водяной знак рисуется средствами android.graphics прямо на изображении страницы. При повторном кодировании в JPEG или PDF метаданные исходного снимка не переносятся.

```
fun Bitmap.withWatermark(text: String, alpha: Int): Bitmap {
    val out = copy(Bitmap.Config.ARGB_8888, true)
    val canvas = Canvas(out)
    val paint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = Color.argb(alpha, 14, 107, 103); textSize = width / 28f
    }
    canvas.rotate(-30f, width / 2f, height / 2f)
    for (y in -height..height * 2 step (paint.textSize * 3).toInt())
        canvas.drawText("$text   ".repeat(6), -width.toFloat(), y.toFloat(), paint)
    return out
}
// повторное кодирование в JPEG/PDF не переносит EXIF исходного снимка
out.compress(Bitmap.CompressFormat.JPEG, 85, stream)
```

Рисунок 17 – Наложение водяного знака

Графический макет приложения

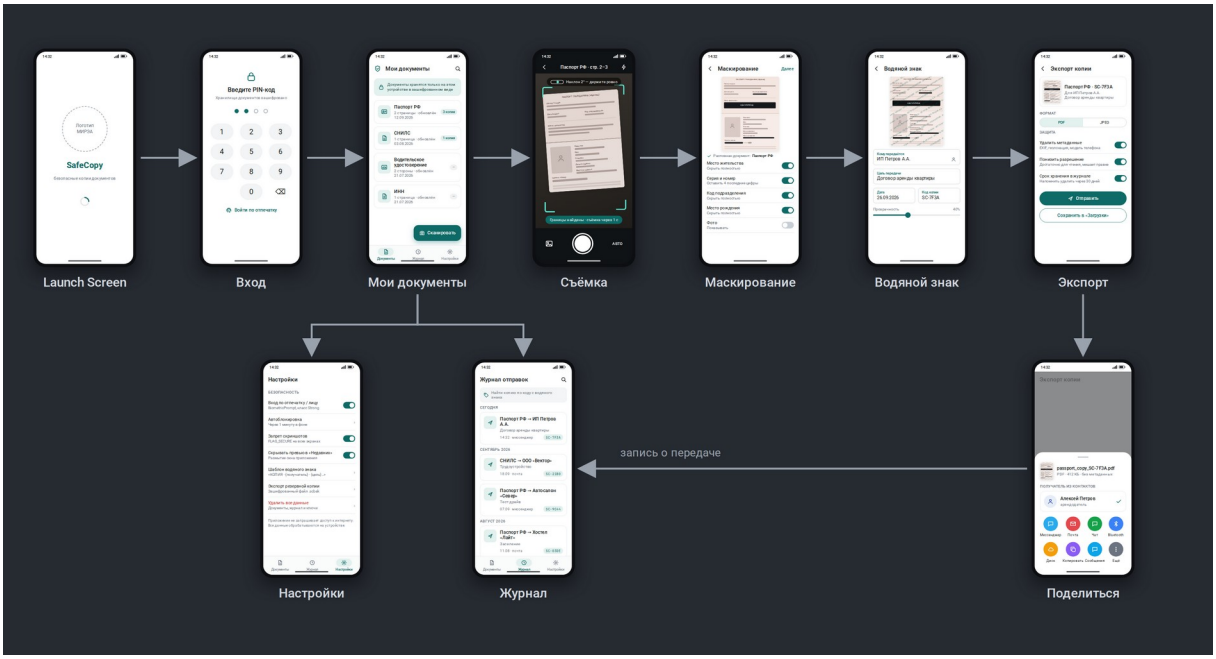


Рисунок 18 – Графический макет приложения